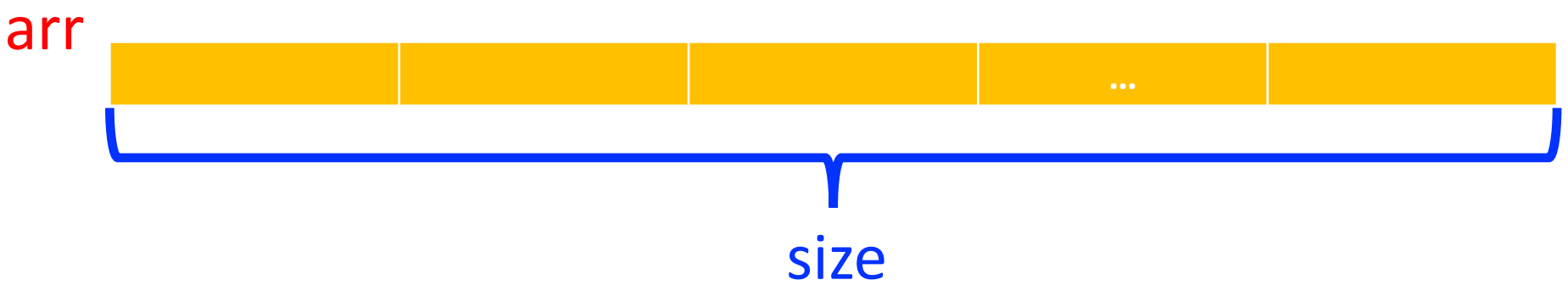
Recursion Exercises

Review Topics

- array
- string
- integer

Function Working with an Array

- For a function working with an array,
  - array's initial address (aka, the address of the first element of the array)
  - the size of the array



Subarray Without the First Element

Cut from the head: subarray without the first element

- initial address of the subarray
- size of the subarray

- Original array: [1, 2, 3, 4, 5] (arr, size = 5)

- Cut from the head:

arr + 1 → [2, 3, 4, 5]

size = 4

Subarray Without the Last Element

Cut from the tail: subarray without the last element

- initial address of the subarray
- size of the subarray

- Original array: [1, 2, 3, 4, 5] (arr, size = 5)

- Cut from the tail:

arr + 1 → [1, 2, 3, 4]

size = 4

Subarray Without the First and Last Element

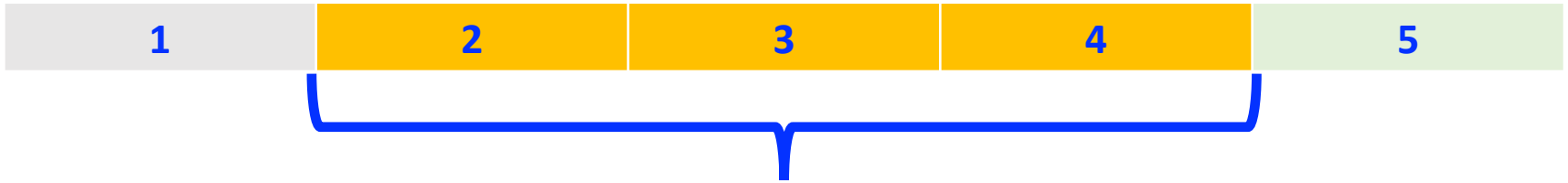
Cut from the head and the tail: subarray without the first and last elements

- initial address of the subarray
- size of the subarray

- Cut from the head and tail:

arr + 1 → [2, 3, 4]

size = 3



Recursion: calculate the sum of an integer array

For example, if an array is {1, 2, 3}, the return is 1 + 2 + 3 = 6.

- function head
- base case
- recursive case

Recursion: find the max of an integer array

For example, if an array is {1, 2, 3}, the return is 3.

- function head
- base case
- recursive case

Recursion: find the max length of a string array

For example, if an array is {“hello”, “hi”, “how”}, the return is 5, the length of “hello”.

- function head
- base case
- recursive case

Recursion: reverse an array of integer

For example, if an array is {1, 2, 3}, after reversal, the array is changed to {3, 2, 1}.

- function head
- base case
- recursive case

Test whether an array is palindrome or not

- An array is a palindrome if it reads the same from left to right as it does from right to left.
  - For example, {1, 2, 1} is palindrome, while {1, 2, 3} is not.

- function head
- base case
- recursive case

Recursive Function Working with a String

Remove the first element

- original string

- substring without the first element

Recursive Function Working with a String: II

Remove the last element

- original string

- substring without the last element

Recursive Function Working with a String: III

Cut from the head and the tail: substring without the first and last elements

- original string

- substring without the first and last elements

Recursive Function: Return a Reversed String

- The return is string.
- For example, given string “abc”, the return is “cba”.

- function head
- base case
- recursive case

Recursive Function: in-place reversal a string

- Return type is void.
- Given string “abc”, after the function, the string is changed to “cba”.

- function head
- base case
- recursive case

Work with an Integer

Cut one digit from the original integer

- Cut the least significant digit (the rightmost one)

or

- Cut the most significant digit (the leftmost one)

Find out the number of digits of an int

- For example, given 125, the return is 3. Given -15, the return is 2.

- function head
- base case
- recursive case

Recursive Leap of Faith

- When you write the **recursive case**, don't try to trace the whole thing in your head.
- **Trust the function**: Assume your function already works for a smaller version of the problem.
- **Your only job**: Figure out how to combine that "smaller result" with the one element you're looking at right now.
  - *Example:* Total Sum = (First Element) + (Sum of the Rest).

Base Case Safety

- Always handle the "empty" or "single" case first.
- **Arrays/Strings:** What happens when size == 0 or size == 1?
- **Integers:** What happens when the number is a single digit. That is, is the number in [-9, 9]?
- **Tip:** If your function crashes, it’s almost always because your base case didn't stop the recursion in time.

Pointer Arithmetic

- In C++, arr + 1 literally moves the starting pointer to the next element.
- If you pass arr + 1 to the next function call, you **must** pass size - 1 as well, or you'll walk right off the end of your memory!

Thinking about Integers

To "cut" digits:

- **Rightmost digit:** num % 10
- **Everything EXCEPT rightmost:** num / 10
- **Leftmost digit:** This is harder! You do not know the number of digits of the number.
- Stick to the rightmost digit approach whenever possible; it’s much cleaner for recursion.

In-Place vs. Return

- **Return type string:** You are building a *new* string. Use the + operator to join characters.
- **Return type void:** You are swapping characters *inside* the original memory. Use std::swap(str[start], str[end]).